Switch statement

- If you have multiple cases depending on different values of the same integer expression, switch is easier to use.
- Syntax:

```
switch (int_expr)
{  case constant_int_value1: stat(s);
   case constant_int_value2: stat(s);
   ...
   default: stat(s);
}
```

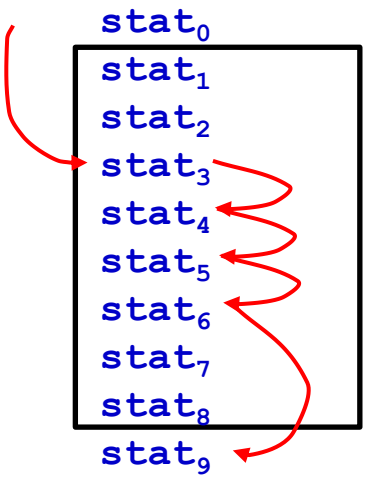
- You may have zero or more statements in each case.

Break statement

- Switch statement actually gathers many statements of several cases.
  - The case labels denote the specific statement from which the execution of this group of statements begins.
  - All statements till the end of the group are executed sequentially.
- To separate the cases, **`break`** statement is used.
  - **`break`** breaks the sequential execution of the statements and immediately jumps to the end of the switch statement.

Break statement

```
stat0;
switch (expr)
{   case value1:
        stat1;
        stat2;
    case value2:
        stat3;
        stat4;
        stat5;
    case value3:
        stat6;
        break;
    case value4:
        stat7;
        stat8;
}
stat9;
```



If `expr` happens to be $value_2$

Switch statement

- Define the days of the week as an enumerated type and display them as strings.

```
enum day_type {MON=1,TUE,WED,THU,FRI,SAT,SUN} day;

scanf("%d", &day);
switch (day)
{   case SUN: printf("Sunday\n"); break;
    case WED: printf("Wednesday\n"); break;
    case TUE: printf("Tuesday\n"); break;
    case THU: printf("Thursday\n"); break;
    case FRI: printf("Friday\n"); break;
    case SAT: printf("Saturday\n"); break;
    case MON: printf("Monday\n"); break;
    default:  printf("Incorrect day\n"); break;
}
```

Switch statement

- Note that without the "break" statement, execution traverses all cases until the end of the switch statement.
- This allows implementation of OR (if you use it properly.
- Eg:

```
switch (number)
{   case 1:
    case 3:
    case 5: printf("Odd number \n"); break;
    case 0:
    case 2:
    case 4: printf("Even number \n"); break;
}
```

Switch statement

- As long as the cases are separated with "break"s, their order is not relevant.
- "default" is optional. If the default case is not specified and none of the cases holds, no statement is executed; this is not an error.
- It is a good practice to put a break even after the last case.

Facts about switch stat. in C-1

- ***1) The expression used in switch must be integral type (int, char and enum).*** Any other type of expression is not allowed.

```c
// float is not allowed in switch
#include <stdio.h>
int main()
{
   float x = 1.1;
   switch (x)
   {
       case 1.1: printf("Choice is 1");
                 break;
       default: printf("Choice other than 1, 2 and 3");
                break;
   }
   return 0;
}
```

Facts about switch stat. in C-2

- ***2) All the statements following a matching case execute until a break statement is reached.***

```c
// There is no break in all cases
#include <stdio.h>
int main()
{
    int x = 2;
    switch (x)
    {
        case 1: printf("Choice is 1\n");
        case 2: printf("Choice is 2\n");
        case 3: printf("Choice is 3\n");
        default: printf("Choice other than 1, 2 and 3\n");
    }
    return 0;
}
```

Facts about switch stat. in C-2

- ***2) All the statements following a matching case execute until a break statement is reached.***

```
// There is no break in some cases
#include <stdio.h>
int main()
{
   int x = 2;
   switch (x)
   {
       case 1: printf("Choice is 1\n");
       case 2: printf("Choice is 2\n");
       case 3: printf("Choice is 3\n");
       case 4: printf("Choice is 4\n");
               break;
       default: printf("Choice other than 1, 2, 3 and 4\n");
                break;
   }
   printf("After Switch");
   return 0;
}
```

Facts about switch stat. in C-3

- ***3) The default block can be placed anywhere.*** The position of default doesn't matter, it is still executed if no match found.

```c
// The default block is placed above other cases.
#include <stdio.h>
int main()
{
    int x = 4;
    switch (x)
    {
        default: printf("Choice other than 1 and 2");
                 break;
        case 1: printf("Choice is 1");
                break;
        case 2: printf("Choice is 2");
                 break;
    }
    return 0;
}
```

Facts about switch stat. in C-4

- ***4) The integral expressions used in labels must be a constant expressions***

```c
// A program with variable expressions in labels
#include <stdio.h>
int main()
{
    int x = 2;
    int arr[] = {1, 2, 3};
    switch (x)
    {
        case arr[0]: printf("Choice 1\n");
        case arr[1]: printf("Choice 2\n");
        case arr[2]: printf("Choice 3\n");
    }
    return 0;
}
```

Facts about switch stat. in C-5

- ***5) The statements written above cases are never executed.*** After the switch statement, the control transfers to the matching case, the statements written before case are not executed.

```c
// Statements before all cases are never executed
#include <stdio.h>
int main()
{
    int x = 1;
    switch (x)
    {
        x = x + 1;  // This statement is not executed
        case 1: printf("Choice is 1"); break;
        case 2: printf("Choice is 2"); break;
        default: printf("Choice other than 1 and 2"); break;
    }
    return 0;
}
```

Facts about switch stat. in C-6

- ***6) Two case labels cannot have same value***

```c
// Program where two case labels have same value
#include <stdio.h>
int main()
{
   int x = 1;
   switch (x)
   {
       case 2: printf("Choice is 1"); break;
       case 1+1: printf("Choice is 2"); break;
   }
   return 0;
}
```

Example 1

- Write a code segment that detects whether a number is divisible by 6.

```
if ((num%2==0) && (num%3==0))
   printf("%d is divisible by 6 \n",num);
```

Example 2

- Write a code segment that detects whether a number is divisible by 3 or 6.

```
if (num%3==0)
   if (num%2==0)
      printf("%d is divisible by 6 \n",num);
   else
      printf("%d is divisible by 3 \n",num);
```

Example 3

- Write a program that reads two real numbers and checks if they are equal.
  (Assume first number is smaller.)

```
#include <stdio.h>
#define EPSILON 0.000000001
float r1, r2;

int main()
{

   scanf("%f %f", &r1, &r2);
   if ((r2-r1)<=EPSILON)
      printf("The numbers are (almost) equivalent");
   return 0;
}
```

Example 4

- Write a program that reads a 3-digit number from the input char-by-char, and displays its square.

```
#include <stdio.h>
char c1, c2, c3;
int num;

int main()
{
   scanf("%c%c%c",&c1,&c2,&c3);
   num  = (c1-'0')*100;
   num += (c2-'0')*10;
   num += c3-'0';
   num *= num;
   printf("%d", num);
   return 0;
}
```

Example 5.1

- Write a program that defines a type for letter grades (AA,BB,CC,DD,F as 4,3,2,1,0), reads a grade and displays corresponding letter grade.

```
#include <stdio.h>

enum grade_type {F,DD,CC,BB,AA};
int main()
{
   enum grade_type g;

   scanf("%d", &g);
   printf("%d", g);
   return 0;
}
```

Example 5.2

- But, we don't want to display a number. We want to see the letter grade.

```
#include <stdio.h>
enum grade_type {F,DD,CC,BB,AA};
int main()
{
   enum grade_type g;

   scanf("%d", &g);
   switch (g)
   {
      case AA: printf("AA \n"); break;
      case BB: printf("BB \n"); break;
      case CC: printf("CC \n"); break;
      case DD: printf("DD \n"); break;
      case F:  printf("F \n");  break;
      default: printf("Invalid \n"); break;
   }
   return 0;
}
```

Example 5.3

- Could we do it without defining an enumerated type?

```
#include <stdio.h>

int main()
{
   int g;

   scanf("%d", &g);
   switch (g)
   {
      case 4:  printf("AA \n"); break;
      case 3:  printf("BB \n"); break;
      case 2:  printf("CC \n"); break;
      case 1:  printf("DD \n"); break;
      case 0:  printf("F \n");  break;
      default: printf("Invalid \n"); break;
   }
   return 0;
}
```

Example 6

- Write a program that reads a character ('a', 'p', or 'v') and radius R. It displays the area or perimeter of a circle with radius R, or the volume of a sphere.

```
#include <stdio.h>
#define PI 3.14  3.141592654
char ch;   float R;

int main()
{
   scanf("%c%f",&ch,&R);
   switch(ch)
   {
      case 'a': printf("Area of circle = %f\n", PI*R*R);
                break;
      case 'p': printf("Perimeter of circle = %f\n", 2*PI*R);
                break;
      case 'v': printf("Volume of sphere = %f\n", (4/3)*(PI*R*R*R));
                break;                            (4.0/3)
      default:  printf("Invalid input\n");
                break;
   }
   return 0;
}
```